

Small Language Model Improvement with Distillation and Neural Cache

DASE7506 MP1 Small Language Model Challenge

WangXinzhe

Student ID u3036801008

24 September 2026

Abstract

This report improves the supplied GPT baseline under the fixed WikiText 2 protocol. The final predictor uses a 6 layer rotary Transformer with RMSNorm and SwiGLU, then adds two core mechanisms: self-distillation from a model trained only on the supplied training split, and a strictly causal neural cache that is reset for every 256 token evaluation window. At 19,200 updates, the architecture without either mechanism obtains 1.55637 validation bits per byte. Distillation alone reaches 1.49356, cache alone reaches 1.49256, and their combination reaches 1.44406. The combined predictor remains inside all measured evaluation limits at 4.484 times baseline CPU time, 1.790 GiB peak RAM, and 25.08 MiB of inference assets.

Main findings

Evidence	Result	Interpretation
Initial baseline	2.07108 BPB	Reference 1,200 update model
Equal target architecture comparison	1.69939 to 1.55637	Architecture improves BPB by 0.14302
Core mechanism ablation	1.55637 to 1.44406	Combined gain is 0.11231 BPB
Resource compliance	4.484x CPU 1.790 GiB 25.08 MiB	All measured limits pass

All model selection and mechanism comparisons use the validation split. A test score is not used in this report as development evidence.

1 Task and contribution

The course fixes the data, BPE 2048 tokenizer, causal 256 token evaluation windows, and the bits per byte scoring rule. The objective is to reduce BPB while keeping CPU scoring time at or below five times the supplied baseline, peak evaluation RAM below 4 GiB, and uncompressed inference assets below 64 MiB. Training length and architecture may change, but all learning must use the supplied training text and model selection must use validation data.

Baseline and improved architecture

Component	Supplied GPT	Improved model
Blocks and width	4 blocks width 128	6 blocks width 288
Attention	4 heads	6 heads with causal SDPA
Position	Learned absolute embedding	Rotary position embedding
Normalization	LayerNorm	RMSNorm
Feed forward network	4x GELU MLP	SwiGLU with width 768
Regularization	None	Residual dropout 0.2
Initialization	Standard deviation 0.02	Depth scaled residual outputs
Parameters	1,088,256	6,565,536 without cache

The larger architecture allocates the available inference budget to representation capacity. RoPE expresses relative position directly in query and key rotations, RMSNorm removes the mean subtraction used by LayerNorm, and SwiGLU adds a learned gate inside the feed forward block. The output projection shares weights with the token embedding. Residual output matrices are initialized with standard deviation 0.02 divided by the square root of twice the depth, which reduces activation growth at initialization.

Measured architecture gain

A 19,200 update run of the supplied architecture gives 1.69939 BPB at the final step. With the same seed, batch size, context, optimizer family, number of updates, and 157,286,400 processed targets, the improved architecture without distillation or cache gives 1.55637 BPB. The 0.14302 BPB reduction isolates the benefit of the architecture and training recipe from additional training data exposure.

2 Core mechanisms

Self distillation

The teacher is the validation-selected checkpoint from the no-cache improved model at update 18,400. It was trained from random initialization on the supplied training split with seed 17. During student training the teacher is fixed in evaluation mode and receives the same causal training prefix as the student. The student objective is $L = 1 - \alpha \times \text{hard label cross entropy} + \alpha \times T^2 \times \text{KL from the teacher distribution to the student distribution}$. The experiments use $\alpha = 0.5$ and temperature $T = 1.0$.

The soft distribution contains information about plausible alternatives beyond the observed next token. This regularizes the student toward a smoother target while retaining a 0.5 weight on the ground-truth label. The teacher is used only during training, so distillation does not change the student's inference parameters, assets, or CPU scoring path.

Causal neural cache

Causal within-window neural cache

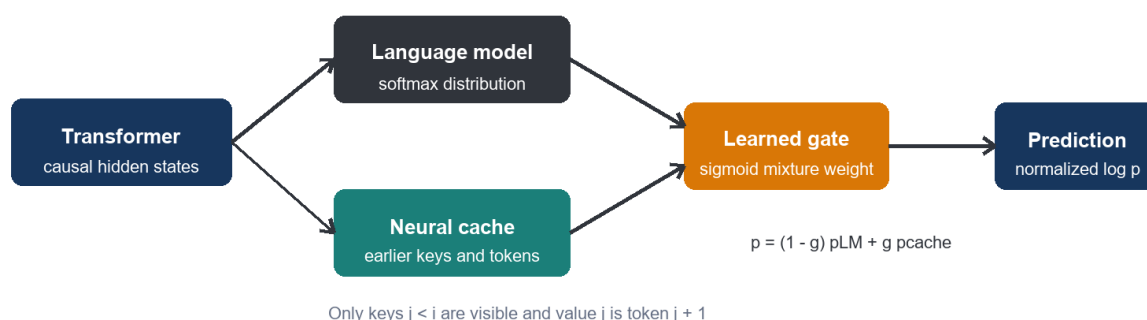


Figure 1 The base distribution and within-window cache are combined by a learned gate

The cache normalizes final hidden states and compares each query with earlier hidden keys. A strict lower triangular mask permits key j only when j is smaller than query position i . Key j stores token j plus 1; therefore the newest token retrieved for query i is token i , which is already in the observed prefix. Similarity weights are divided by a learned temperature bounded between 0.05 and 0.50 and accumulated into a vocabulary distribution.

A learned sigmoid gate mixes the language model and cache probabilities. Its bias starts at minus 2.5 so early training relies mostly on the base model. The cache is local to one forward call and contains no persistent state. It resets between windows and examples, satisfying the course rule that evaluation windows are independent and no state crosses a boundary.

3 Experimental protocol

Data and scoring

All runs use the supplied WikiText 2 text and fixed BPE 2048 tokenizer. Training samples random 257 token spans and predicts the final 256 tokens. Validation scoring covers 376,599 targets and 1,148,007 UTF 8 bytes in independent causal windows. BPB is summed negative log base 2 probability divided by the raw byte count. Development decisions use validation only.

Controlled settings

Setting	Value
Random seed	17
Updates and batch size	19,200 updates batch 32
Targets per run	157,286,400
Context and vocabulary	256 tokens 2,048 tokens
Optimizer	AdamW beta1 0.9 beta2 0.95 weight decay 0.1
Learning rate	0.001 peak 100 update warmup cosine decay to 0.0001
Gradient clipping	Global norm 1.0
Training precision	FP32 on NVIDIA GeForce GTX 1660 Ti
Distillation	alpha 0.5 temperature 1.0

Experiment matrix

Run	Distillation	Cache	Selection step	Best validation BPB
No mechanisms	No	No	18,400	1.55441
Cache only	No	Yes	16,000	1.49206
Distillation only	Yes	No	19,200	1.49356
Both mechanisms	Yes	Yes	19,200	1.44406

The four runs differ only in the two core mechanisms. Main causal claims use the final update for all four runs, so every comparison has identical processed targets. Validation-selected results are reported separately because their selected steps differ.

4 Main results

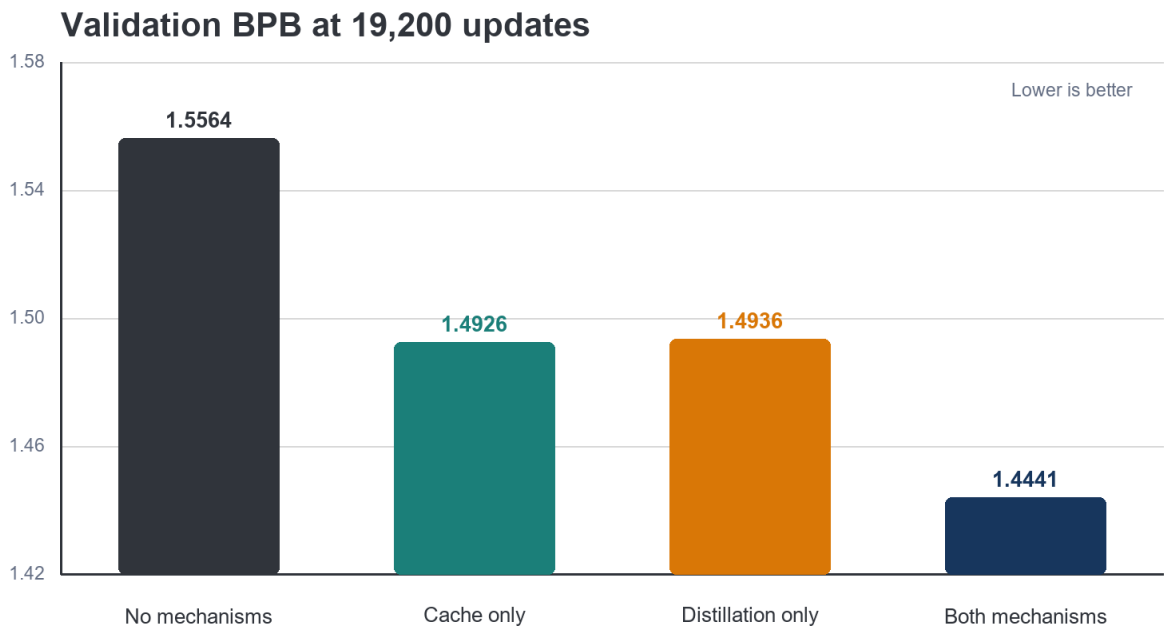


Figure 2 Fixed-step comparison with equal training targets

Distillation	Cache	BPB at 19,200	Gain from no mechanisms
No	No	1.55637	reference
No	Yes	1.49256	0.06380
Yes	No	1.49356	0.06281
Yes	Yes	1.44406	0.11231

Both mechanisms produce similar standalone gains. Cache lowers BPB by 0.06380 without distillation, while distillation lowers it by 0.06281 without cache. Combining them lowers BPB by 0.11231, a 7.22 percent reduction from the no-mechanism architecture. The final validation BPB of 1.44406 is 0.62703 below the initial 1,200 update course baseline, although that larger comparison also includes more capacity and training.

Interaction between mechanisms

Cache adds 0.04950 BPB of improvement when distillation is already active, and distillation adds 0.04851 when cache is already active. These conditional gains remain substantial but are smaller than the standalone gains. The positive 0.01430 BPB interaction term indicates diminishing returns: both methods improve the probability assigned to plausible continuations, so part of their benefit overlaps rather than adding independently.

5 Training dynamics and mechanism analysis

Validation BPB during training

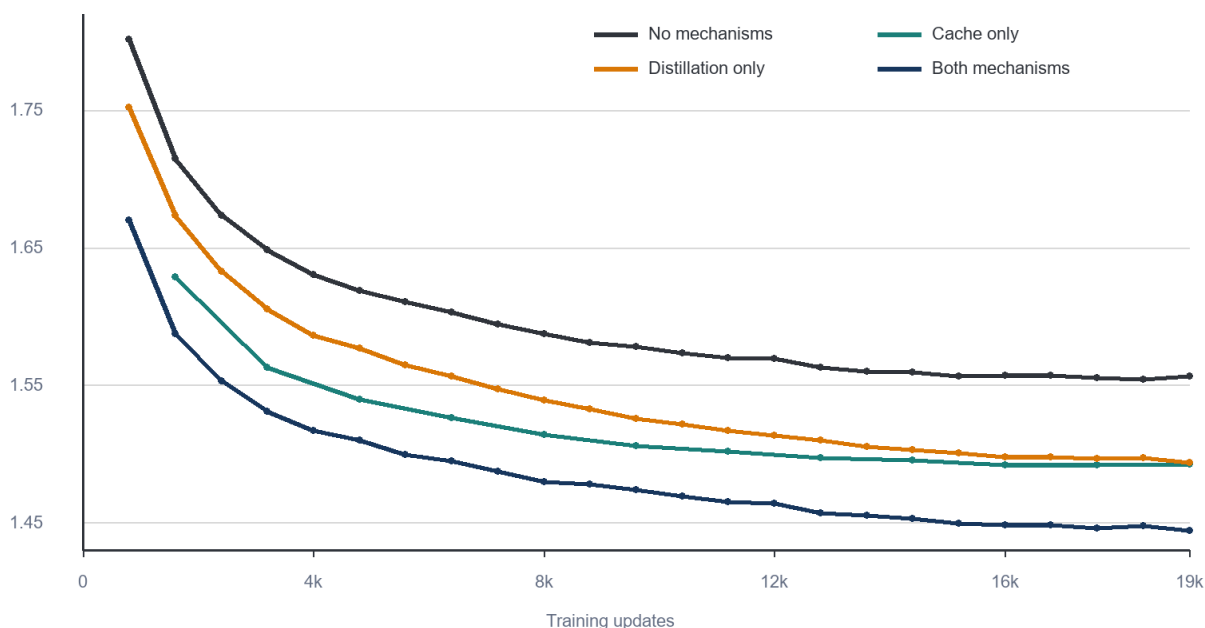


Figure 3 Validation trajectories for the four controlled runs

The cache improves the model throughout training rather than only at the final checkpoint. At update 12,800, cache-only BPB is 1.49750 compared with 1.56318 for the no-mechanism run. At the same update, distillation-only BPB is 1.51013 and the combined model is 1.45706. The combined curve continues to improve through update 19,200, supporting selection of the final checkpoint.

Why the cache helps

WikiText contains repeated names, entities, and topical vocabulary inside a 256 token window. The parametric head must represent these patterns in fixed weights, while the cache can directly reuse a recent token after matching a similar hidden context. Because the gate is learned at every position, the model can suppress the cache when repetition is unhelpful.

Why distillation helps

A one-hot label treats every incorrect token equally. The teacher distribution distinguishes reasonable alternatives from implausible ones and stabilizes training with a smoother target. The teacher and student share the same architecture, so this is self-distillation rather than compression from a larger model. Its gain is therefore best interpreted as optimization and regularization rather than transfer of external knowledge.

Failure modes and tradeoffs

The cache can overemphasize accidental repetition and adds an $O(L^2)$ similarity matrix plus a vocabulary scatter. Distillation nearly doubles training forward computation because the teacher must score every batch. The observed overlap between their gains also means that further generations of self-distillation may yield smaller improvements.

6 Efficiency and compliance

Evaluation resource limits

Constraint	Measured result	Course limit	Status
CPU FP32 scoring time	38.95 s 4.484x baseline	at most 5x	Pass
Peak CPU evaluation RAM	1.790 GiB	at most 4 GiB	Pass
Inference assets	25.08 MiB	at most 64 MiB	Pass

CPU time is measured on the same four-thread machine and split for both models: 8.686 seconds for the supplied baseline and 38.952 seconds for the final predictor. The resulting 4.484 times ratio leaves about 10.3 percent of the permitted timing budget. RAM and assets have wider margins. The timing result is the tightest constraint and should be reproduced on the submitted software stack.

Training and search cost

Run	Direct train time	Processed targets	GPU peak reserved
Long supplied baseline	11.6 min	157,286,400	0.68 GB
No mechanisms teacher	62.1 min	157,286,400	1.82 GB
Cache only	71.0 min	157,286,400	2.20 GB
Distillation only	80.9 min	157,286,400	2.18 GB
Both mechanisms	85.1 min	157,286,400	2.51 GB

The recorded runs account for 5.32 hours of direct training, including the 1,200 update initial CPU baseline. The two distilled runs reuse the same no-mechanism teacher, but reuse does not erase its 3,726.8 second training cost. For checkpoint ancestry, the final model therefore depends on the teacher's 157,286,400 targets plus its own 157,286,400 targets.

Data and causal compliance

The teacher and all students learn only from the supplied training text. Validation is used for checkpoint selection; no validation or test targets are cached. Predictions at position i use only the prefix through i , and all cache tensors are created inside the current forward call. The tokenizer, data files, evaluator, context length, and scoring rule remain unchanged.

7 Reproduction and conclusion

Final training command

```
python train.py --implementation student --config configs\student_long_cache.json --device cuda --precision fp32 --seed 17 -  
-steps 19200 --batch-size 32 --eval-every 800 --teacher-checkpoint runs\student-long-s17\checkpoint.pt --distill-alpha 0.5 -  
-distill-temperature 1.0 --run-dir runs\student-cache-distill-s17
```

CPU validation and resource check

```
python evaluate.py --checkpoint runs\student-cache-distill-s17\checkpoint.pt --device cpu --precision fp32 --split  
validation --output runs\student-cache-distill-s17\validation_cpu_fp32.json  
python -m unittest discover -s tests -v
```

After freezing the method, the complete test score should be produced once with the same checkpoint, CPU device, and FP32 precision by changing only the split to test. The test result must not be used to revise the method.

Checkpoint identity

Artifact	SHA256 or identifier
Final checkpoint	9e7ae095e6a3da3589cf3e6857b058e08d7bac00a7022d9e82d50bb00e38a44c
Teacher checkpoint	413312764929069f79242474c2cc0012cbab2afe190246df0b71a2d1d507e313
Student implementation	68a9fdb1c678ec5ad0306b02115b8accb15fd0577f4dad9ad527d65cf77acec1
Protocol	7506-mp1-wt2-v2

Conclusion

The controlled 2 by 2 experiment supports both proposed mechanisms. Distillation and the causal cache each reduce validation BPB by about 0.063 when used alone, and the combined model reaches 1.44406 at equal training targets. The mechanisms have partially overlapping effects, but each still contributes about 0.049 BPB in the presence of the other. The final predictor satisfies the measured CPU time, memory, and asset limits, with CPU timing as the main remaining margin risk.

AI assistance disclosure

Substantive AI assistance was used to discuss architecture and experiment choices, implement optional training mechanisms, and draft and format this report. I verified the configurations, checkpoint metadata, numerical results, resource measurements, and compliance claims against the repository code and experiment logs.

References

- 1 DASE7506 MP1 Small Language Model Challenge guide and code README 2026
- 2 Merity S Xiong C Bradbury J and Socher R Pointer Sentinel Mixture Models 2016
- 3 Vaswani A et al Attention Is All You Need 2017
- 4 Su J et al RoFormer Enhanced Transformer with Rotary Position Embedding 2021
- 5 Zhang B and Sennrich R Root Mean Square Layer Normalization 2019
- 6 Shazeer N GLU Variants Improve Transformer 2020
- 7 Hinton G Vinyals O and Dean J Distilling the Knowledge in a Neural Network 2015
- 8 Grave E Joulin A and Usunier N Improving Neural Language Models with a Continuous Cache 2017